

1. What is Unit Testing?

Definition:

Unit Testing is the process of testing the **smallest testable unit** (method, function, or class) of an application independently to verify that it works correctly.

Example:

Testing the Add() method of a Calculator class.

```
public int Add(int a, int b)
{
    return a + b;
}
```

Features

- Tests a single method or function.
- Performed by developers.
- Fast and easy to execute.
- Uses mock objects for external dependencies.

2. Difference Between Unit Testing and Functional Testing

Unit Testing	Functional Testing
Tests a single unit (method/function)	Tests complete application functionality
Done by developers	Done by testers/QA
Uses mocks/stubs if needed	Uses real components
Fast	Comparatively slower
Verifies code logic	Verifies business requirements

3. Types of Testing

a) Unit Testing

Tests individual methods or classes independently.

Example: Testing the Add() method.

b) Functional Testing

Checks whether the application behaves according to business requirements.

Example: Verifying the Calculator application performs all operations correctly.

c) Automated Testing

Tests are executed automatically using tools like NUnit, Selenium, or JUnit.

Benefits:

- Saves time
- Reduces manual work
- Can be executed repeatedly

d) Performance Testing

Measures application speed, response time, scalability, and stability under load.

Example: Checking how many users a website can handle simultaneously.

4. Benefits of Automated Testing

- Saves testing time.
- Detects bugs early.
- Reduces manual effort.
- Improves software quality.
- Supports Continuous Integration/Continuous Deployment (CI/CD).
- Makes regression testing easier.
- Provides consistent and repeatable results.

5. Loosely Coupled & Testable Design

Loosely Coupled Design

A loosely coupled design means one class **does not depend directly on another concrete class**. Instead, it depends on interfaces or abstractions.

Benefits

- Easy to test
- Easy to maintain
- Easy to replace dependencies
- Supports mocking during unit testing

Example

Tightly Coupled:

```
public class Calculator
{
    Database db = new Database();
}
```

Loosely Coupled:

```
public interface IDataService
{
    int GetValue();
}
```

```
public class Calculator
{
    private readonly IDataService service;

    public Calculator(IDataService service)
    {
        this.service = service;
    }
}
```

6. First NUnit Program (Calculator Addition)

```
using NUnit.Framework;
using ClassLibrary;
namespace CalculatorTests
{
    public class CalculatorTests
    {
        private Calculator calculator;

        public void Setup()
        {
            calculator = new Calculator();
        }

        public void TearDown()
        {
            calculator = null;
        }

        [TestCase(2,3,5)]
        [TestCase(10,20,30)]
        [TestCase(-5,5,0)]
        public void AddTest(int a,int b,int expected)
        {
            int actual = calculator.Add(a,b);

            Assert.That(actual, Is.EqualTo(expected));
        }
    }
}
```

7. NUnit Attributes

[TestFixture]

Marks a class as a test class.

```
[TestFixture]
```

```
public class CalculatorTests
```

```
{
```

```
}
```

Marks a method as a test method.

```
[Test]
```

```
public void AddTest()
```

```
{
```

```
}
```

```
public void Setup()
```

```
{
```

```
    calculator = new Calculator();
```

```
}
```

Purpose: Initializes objects before each test.

[TearDown]

```
public void TearDown()
```

```
{
```

```
    calculator = null;
```

```
}
```

Purpose: Cleans up resources after each test.

```
[Ignore("Work in progress")]
```

```
public void TestMethod()
```

```
{
```

```
}
```

Purpose: Prevents execution of a test that is not ready.

8. Parameterized Test Cases ([TestCase])

Instead of writing multiple test methods, one method can run with different input values.

```
[TestCase(2,3,5)]
```

```
[TestCase(10,20,30)]
```

```
[TestCase(-5,5,0)]
```

```
public void AddTest(int a,int b,int expected)
```

```
{
```

```
    int actual = calculator.Add(a,b);
```

```
    Assert.That(actual, Is.EqualTo(expected));
```

```
}
```

Benefits

- Reduces duplicate code.
- Improves readability.
- Allows multiple input combinations.
- Easier to maintain.

9. Assert.That()

Used to compare the actual result with the expected result.

```
Assert.That(actual, Is.EqualTo(expected));
```

If both values match, the test passes; otherwise, it fails.